



SMART CONTRACT SECURITY ASSESSMENT

PREPARED FOR

EDENA TOKEN



hello@interfi.network



interfinetwork



www.interfi.network

FOR EDENA

Date of Report	August 25, 2025
Implementation Code	0xd5D2FCcB74Ae15D288ebA5D113f8911c44B9De8d
Proxy Code	0x618e80a3F00a3907CC021187e9beb73AE4D658A6
Platform	Ethereum
Website	http://edenatoken.com
X	https://twitter.com/edenaofficial
Language	Solidity
Methodology	Automated Review, Unit Tests, Manual Review
Auditor	InterFi

- Disclaimer: Smart contracts deployed on blockchains are inherently exposed to potential exploits, vulnerabilities, and security risks. Blockchain and cryptographic technologies are emerging and carry ongoing uncertainties. Please review the full audit report for detailed insights into risk severity, vulnerabilities, and audit scope limitations.
- Centralization Warning: Centralized privileges—regardless of intent or access control—introduce elevated risks to contract security and user trust.
- KYC Advisory: The project lacks verified third-party KYC of its owners, team, or deployers. Without independent KYC, transparency and accountability are reduced, increasing the risk of fraud or rug pulls.
- Verification: Verify this report: <https://www.github.com/interfinetwork>

TABLE OF CONTENTS

FOR EDENA	1
TABLE OF CONTENTS	3
1. SUMMARY	4
1.1 Summary of Findings	4
1.2 Resolution Status	4
1.3 System Overview	5
1.4 Files in Scope	5
1.5 Out-of-Scope Assumptions	6
2. METHODOLOGY	7
2.1 Audit Objectives	7
2.2 Methodologies	7
2.3 Risk Categorization	8
2.4 Resolution Status Definitions	9
3. FINDINGS	10
4. CENTRALIZATION	23
4.1 Privileged Functions	23
5. DISCLAIMER	27
5.1 Confidentiality	27
5.2 No Financial Advice	27
5.3 Technical Disclaimer	27
5.4 Timeliness & Accuracy	28
5.5 Third-Party Links	28
6. ABOUT	29
6.1 Connect with Us	29

1. SUMMARY

The audit resulted in the identification of issues across a range of severity levels, including logic flaws, access control oversights, and design inconsistencies. All high-impact findings were communicated to the development team with clear recommendations for remediation. Where applicable, the team has confirmed implementation of fixes or provided justifications for design choices.

1.1 Summary of Findings

Severity	Count
Critical	1 / FIXED
Major	1 / FIXED
Medium	5 / SOME PENDING
Minor	5 / FIXED
Unknown	1 / ACKNOWLEDGED
Centralization	2 / ACKNOWLEDGED

1.2 Resolution Status

Status	Count
Fixed	8
Partially Fixed	2
Acknowledged	3
Pending	2

1.3 System Overview

This system is a decentralized protocol comprising a suite of smart contracts. These contracts collectively define the rules, permissions, and operational workflows for managing on-chain assets, executing user interactions, and enforcing protocol-level logic. Smart contracts in this context are self-executing code units that autonomously manage the state and behavior of digital assets based on predefined conditions. The protocol utilizes these contracts to enable key functionalities such as:

- Ownership and access control enforcement
- Permission and role-based actions
- Data storage and updates
- Event logging and auditability
- Batch processing and collection management

1.4 Files in Scope

InterFi was engaged by Edena to perform a security audit of the smart contracts. The audit scope was strictly limited to the files explicitly listed under the “Files in Scope” section. No other files or components were reviewed unless otherwise stated.

File	Path	Notes
Edena	Edena.sol	

1.5 Out-of-Scope Assumptions

The following components and assumptions were explicitly excluded from this audit:

- Frontend or backend integration logic.
- Off-chain components, scripts, or oracles.
- External contracts or libraries unless explicitly stated.
- Compiler-level or EVM-specific behavior outside the contract's scope.
- Governance or tokenomics-related decisions not implemented in code.
- All third-party dependencies as discussed in findings.

2. METHODOLOGY

2.1 Audit Objectives

This audit aims to ensure that the smart contract system is predictable, and behaves as intended under normal conditions. Primary audit objectives are to:

- Identify potential vulnerabilities or logic errors in the implementation.
- Evaluate adherence to best practices in smart contract development.
- Assess the correctness of access controls and permission systems.
- Recommend remediations or enhancements for improved security and performance.

2.2 Methodologies

The audit follows a layered security approach using both automated tools and manual techniques. We review the contracts for functional correctness, exploitability, and adherence to smart contract best practices:

Type	Tools & Techniques
Manual Code Review	Line-by-line analysis to check logic, permissions, and edge cases
Automated Analysis	Tools like Slither, MythX, or custom linters to catch known patterns
Static Analysis	Identification of bugs without executing the code (compile-time checks)
Unit Test Inspection	Evaluation of existing test coverage, assumptions, and potential false positives/negatives (if applicable)
Architecture Review	Mapping of privileged roles, callable paths, and contract interdependencies (if applicable)

2.3 Risk Categorization

Each issue identified during the audit is assigned a severity level based on its potential impact, exploitability, and likelihood of real-world abuse. These categories help prioritize remediation efforts:

Risk Severity	Definition
Critical	Represents a severe vulnerability that may result in complete contract compromise, such as asset theft, permanent loss of functionality, or unrestricted access. These issues are often easily exploitable and require immediate resolution.
Major	Indicates significant risk that can affect core contract behavior, enable unauthorized operations, or create unintended financial exposure. While not as urgent as critical risks, they should be remediated promptly.
Medium	These are moderate-level risks that may become exploitable under specific conditions. They often relate to logic errors, insufficient validation, or architectural oversights that could escalate over time.
Minor	Denotes issues that have low security impact but may degrade code quality, performance, or maintainability. These include inefficiencies, style violations, or redundant logic. Fixes are recommended for robustness.
Unknown	Risks where the severity cannot be confidently determined due to limited context, external dependencies, or ambiguous design intent. It is advised to treat these conservatively and address them proactively.
Centralization	Any function controlled by a single privileged role is treated as a critical risk, regardless of its purpose, due to the potential for misuse, override, or total asset control.

2.4 Resolution Status Definitions

All identified issues are also assigned a resolution status, indicating the current handling and response from the development team:

Status	Definition
 Fixed	The issue has been remediated and verified as resolved during the re-audit or final check.
 Partially Fixed	The issue has been partially mitigated, but remnants or related concerns may still exist. Further attention may be required.
 Acknowledged	The development team has accepted the finding but opted not to implement a fix.
 Pending	The issue remains unresolved at the time of publication. It poses a potential risk and should be addressed.

3. FINDINGS

01	Retroactive Snapshot Tampering Allows Post-Facto Balance Forgery
Severity	Critical
Description	snapshot() records balances at block.timestamp into snapshotBalances[snapshotId][addr] and populates snapshotAddresses[snapshotId]. However, owner can later call: addAddressToSnapshot(address, snapshotId) to write the address's current live balance into an old snapshot (snapshotId <= currentSnapshotId). There's no guard that written balance equals balance at the snapshot time, nor any frozen snapshot invariant. This rewrites history. Any distribution, voting, airdrop, or accounting that relies on snapshot balances can be manipulated <u>after the fact</u>: ▪ Inflate a friendly address's snapshot balance. ▪ Deflate others (by omitting them, then adding only selected addresses). ▪ Rewrite earlier snapshots using balances that occur much later.
Recommendation	Make snapshots immutable. Use <i>OpenZeppelin ERC20Snapshot</i> which internally handles balances at snapshot time, or store per-account snapshot on transfer/mint/burn deltas so balanceOfAt is reconstructed from historical diffs.
Status	Fixed
Comment	addAddressToSnapshot / addAddressesToSnapshot were removed. Once snapshotCompleted[sid] is true, write-paths are blocked.

02Unbounded Freeze Horizon	
Severity	Major
Description	Any address in approveArr (set by owner) can call: lock(holder, value, releaseTime) This locks tokens already owned by holder without the holder’s consent. releaseTime can be arbitrarily far in the future, effectively <u>freezing funds</u>.
Recommendation	Introduce a lock policy: max lock duration, per-account quotas, allow-list of lockable addresses
Status	● Fixed
Comment	Added MAX_LOCK_DURATION = 1460 days and enforced in _lock.

03 Snapshot Enumeration & Gas Blow-Up [DoS]	
Severity	Medium
Description	<code>snapshot()</code> iterates every address ever seen (<code>addressList</code>) and pushes non-zero balance addresses into <code>snapshotAddresses[snapshotId]</code>. <code>addressList</code> grows unbounded (auto-register on <code>transfer</code> / <code>transferFrom</code>), and there's no pruning. On mature tokens, <code>addressList.length</code> can be tens of thousands, making <code>snapshot()</code> revert from out-of-gas.
Recommendation	Use <i>OpenZeppelin</i> <code>ERC20Snapshot</code> which internally handles balances at snapshot time.
Status	● Fixed

04 Transfer Gas-Griefing via Large timelockList	
Severity	Medium
Description	With <code>autoUnlockEnabled = true</code> (default), each <code>transfer/transferFrom</code> on an address with any locks performs a linear scan of all its locks (while (<code>i < timelockList[holder].length</code>)). If many non-expired locks exist, transfers pay $O(N)$ cost every time and can hit gas limits.
Recommendation	Keep a min-heap / sorted list by <code>releaseTime</code> to stop scanning when first non-expired is encountered. Introduce a cap on per-account concurrent locks.
Status	● Partially Fixed
Comment	Sorted-locks attempt + optimized <code>_autoUnlock()</code> added, but a logic bug reintroduces stuck expired locks. See 04-01

04-01Expired locks can remain stuck due to reordering	
Severity	Medium
Description	Users remain artificially locked even after expiry until multiple manual calls (or never, if they rely on auto). <code>autoUnlock()</code> assumes index <code>0</code> is always the earliest and repeatedly removes index <code>0</code> while it's expired. But <code>_removeLock()</code> swaps the last element into <code>idx</code> then <code>pop()</code>. That breaks the sorted order. After the first removal, index <code>0</code> may now be a <i>late</i> lock, the loop stops, leaving other already-expired locks at higher indices still locked.
	Preserve order on removal: <pre>function _removeLock(address holder, uint256 idx) internal { LockInfo[] storage arr = timelockList[holder]; uint256 amount = arr[idx]._amount; for (uint256 i = idx; i + 1 < arr.length; i++) arr[i] = arr[i + 1]; arr.pop(); lockedAmount[holder] -= amount; emit Unlock(holder, amount, msg.sender); }</pre>
Recommendation	
Status	Pending

05 Freeze List Only Checks Sender	
Severity	Medium
Description	A frozen account cannot send, but it can receive tokens: notFrozen is not applied to to. This may be desired, but commonly, freezes prevent both sending and receiving to stop unwanted inflows or grief via dusting.
Recommendation	Add require(!frozenAccount[to], "recipient frozen"); or define intended semantics.
Status	● Partially Fixed
Comment	transferWithLock path still allows sending to a frozen recipient. See 05-01

05-01 Freeze bypass via transferWithLock	
Severity	Medium
Description	Base transfers check both parties with notFrozen. transferWithLock* relies on _transfer (which now only checks pause + unlocked balance), so a frozen recipient can still receive tokens via this path.
Recommendation	Potential fix (add to Base and Easy too): <pre>function transferWithLock(...) public onlyApproved nonReentrant returns (bool) { require(!frozenAccount[holder], "recipient frozen"); require(!frozenAccount[msg.sender], "sender frozen"); ... }</pre>
Status	Pending

06 Missing Events	
Severity	Minor
Description	Gas blow-ups on critical admin actions; reduced transparency of privilege changes. No event emitted on add/remove → off-chain monitoring is harder (security ops blind spot).
Recommendation	Emit events
Status	● Fixed
Comment	Added events.

07	No Public pause() / unpause()
Severity	Minor
Description	whenNotPaused guards transfer/transferFrom, but no external pause()/unpause() exist, so pausing is currently inoperative.
Recommendation	Either fully implement pause()/unpause() and enforce pause in _beforeTokenTransfer for all transfers; or remove Pausable to avoid false expectations.
Status	● Fixed
Comment	Added pause()/unpause().

08	Confusing Lock Accounting
Severity	Minor
Description	<code>getLockedBalance(holder)</code> returns <code>lockedAmount[holder]</code> which only decreases when an unlock happens; expired-but-unclaimed locks remain counted. Meanwhile <code>getLockTotal</code> sums only unexpired locks.
Recommendation	Expose a clear <code>getLockedUnexpired()</code> and <code>getLockedIncludingExpired()</code> pair
Status	● Fixed
Comment	Separated <code>getLockedIncludingExpired</code> vs <code>getLockedUnexpired</code>

09	Duplicates Possible in snapshotAddresses
Severity	Minor
Description	No deduplication when adding addresses post snapshot; same address can appear multiple times in snapshotAddresses[snapshotId], while snapshotBalances is last-write-wins.
Recommendation	Check isAddressInSnapshot (or mapping flag) before push.
Status	Fixed
Comment	Batch writer checks snapshotBalances[sid][acct] == 0 before pushing, so no duplicates while paused.

10	Potential front-running
Severity	Minor
Description	A privileged approved locker can front-run a user’s outgoing transfer by calling <code>lock(holder, value, releaseTime)</code> first, causing the transfer to revert (Transfer amount exceeds unlocked balance) or forcing the holder’s funds to remain illiquid. Users (or block producers) can move balances around <code>snapshot()</code> calls so they are included/excluded or sized favorably, since the snapshot writes current live balances. If post-facto additions are allowed, timing/ordering further amplifies influence.
Recommendation	Restrict or further scope the locking authority and add guardrails to reduce front-run griefing (e.g., max duration, per-account caps, optional holder-signed consent for third-party locks, or a short notice period before a new lock becomes effective). For snapshots, pre-announce the snapshot height, make snapshots immutable, or migrate to <code>ERC20Snapshot</code> which captures balances at the snapshot instant and avoids post-facto mutation.
Status	● Fixed
Comment	Removing <code>lock()</code> eliminates third-party locking of someone else’s existing balance, plus added cooldown and max duration guardrails. Snapshot is taken while paused, so timing games during processing are neutralized.

11	Direct & Indirect Dependencies
Severity	Unknown
Description	Smart contract stack relies on multiple external/third-party components and patterns, <i>OpenZeppelin</i> upgradeable modules (<i>Initializable</i> , <i>OwnableUpgradeable</i> , <i>ReentrancyGuardUpgradeable</i> , <i>ERC20Upgradeable</i>), <i>UUPS/ERC1967Utils</i> , and environment assumptions (validator ordering, mempool behavior). Any vulnerability, divergence from audited upstream code, or upstream upgrade/migration can affect security and liveness.
Recommendation	Inspect third party dependencies regularly, and mitigate severe impacts whenever necessary
Status	Acknowledged

4. CENTRALIZATION

Centralization is one of the leading causes of smart contract-related asset losses. When a contract assigns critical powers to a privileged role—such as an `owner`, `admin`, or designated `controller`—the associated risk becomes elevated, especially if that role is tied to a single externally owned account (EOA). In many cases, privileged roles serve operational or safety functions, including:

- Emergency Controls: Ability to `pause()` the contract during active threats or bugs.
- Contract Configuration: Updating key addresses, thresholds, or operational variables post-deployment.

4.1 Noteworthy Privileged Functions

- `__Approvable_init`
- `addApproveArr`
- `removeApproveArr`
- `_authorizeUpgrade`
- `setLockupDays`
- `setAutoUnlockEnabled`
- `registerAddress`
- `unregisterAddress`
- `lock`
- `transferWithLock`
- `transferWithLockEasy`
- `transferWithLockBase`
- `unlock`
- `manualUnlock`
- `snapshot`

- addAddressToSnapshot
- addAddressesToSnapshot
- freezeAccount
- unfreezeAccount

01	Centralized Privileges
Severity	Centralization
Description	EOAs with control can be compromised via phishing, private key leakage, or insider threats. Malicious or negligent use of privileges can lead to - token supply manipulation, disruption of trading via pausing, arbitrary fee changes or wallet exclusions, asset seizures or rerouting, etc. <u>4.1 Privileged Functions</u>
Recommendation	Using Multi-Signature Wallets: Assign privileged roles to a multi-sig contract requiring signatures from multiple trusted parties. This reduces the impact of any single compromised key. Time-Locked Functions: Introduce delays before executing sensitive operations, allowing time for community review or cancellation. Role Revocation or Transfer: If privileges are no longer needed post-deployment, renounce them or migrate them to DAO governance. Secure Key Management: Any private keys associated with privileged roles must be protected using hardware wallets, secret sharing schemes, or offline signing protocols.
Status	Acknowledged

5. DISCLAIMER

InterFi Network provides professional smart contract audits for blockchain-based codebases (commonly known as smart contracts). This audit assessed the reviewed contract(s) for common vulnerabilities, centralization risks, and logic flaws. However, no audit can guarantee the complete absence of bugs or vulnerabilities. This report does not constitute a security guarantee, endorsement, or assurance of business model soundness or legal compliance.

The review is limited strictly to the source code and its logic as provided, and does not extend to compiler behavior, off-chain components, or external integrations. Due to the evolving nature of blockchain technology and associated risks, users should understand that all materials, including this audit report, are provided strictly on an “as is”, “as available”, and “with all faults” basis.

5.1 Confidentiality

This report is confidential and intended solely for the client. It may not be disclosed, reproduced, or relied upon by third parties without prior written consent from InterFi Network. All terms, including confidentiality, liability limitations, and scope, are governed by the audit agreement.

5.2 No Financial Advice

This report is not financial, investment, tax, legal, or regulatory advice. It should not be relied upon for making investment decisions or assessing the value, viability, or safety of any token, product, or platform. No part of this document should be interpreted as an endorsement or recommendation. InterFi Network accepts no liability for any actions taken based on this report.

5.3 Technical Disclaimer

InterFi disclaims all warranties—express, implied, or statutory—including merchantability, fitness for a particular purpose, title, and non-infringement. We do not guarantee that the reviewed contracts are error-free, fully secure, or meet any specific requirements. Audit results may contain false positives or negatives, and findings are subject to the context and limitations of the review scope.

5.4 Timeliness & Accuracy

Audit results reflect the state of the code at the time of review. InterFi makes no commitment to update findings after publication. We do not warrant the accuracy, completeness, or timeliness of information delivered via this report.

5.5 Third-Party Links

This report may contain references or links to external websites and social media accounts. InterFi Network is not responsible for the content or operation of third-party platforms and assumes no liability for actions taken based on their content.

6. ABOUT

InterFi Network is a leading provider of intelligent blockchain solutions, offering secure, scalable, and production-ready smart contract services. Our team specializes in the development, testing, and auditing of smart contracts across a wide range of blockchain ecosystems.

We have delivered:

- 300+ smart contract systems developed
- 2,000+ smart contracts audited
- 500,000+ lines of code reviewed and analyzed

Our technical expertise spans multiple languages including:

- Solidity for EVM-compatible chains (Ethereum, BNB Chain, Polygon, Avalanche, Cronos, Fantom, Velas, Metis, and more)
- Move for next-generation platforms such as Sui and Aptos
- Rust for advanced ecosystems like Solana, Near, and Cosmos SDK-based chains

6.1 Connect with Us

InterFi Network is driven by a multidisciplinary team of engineers, developers, UI/UX specialists, and blockchain researchers. The core team consists of 3 senior members supported by 4+ expert contributors across code auditing, tooling, and protocol design.

- Website: interfi.network
- Email: hello@interfi.network
- GitHub: github.com/interfinetwork
- Telegram (Engineering): [@interfiaudits](https://t.me/interfiaudits)
- Telegram (Onboarding): [@interfisupport](https://t.me/interfisupport)

THANK YOU